

* Linker Script

- Linker Script is a text file which explains how different sections of the object files should be merged to create an output file.
- Linker and locator combination assign unique absolute address to different sections of the output files by referring address information mentioned in the linker script.
- Linker script also include the code and data memory address and size information.
- Linker script are written using the GNU linker command language.
- GNU linker script has the file extension of .ld
- You must supply linker script at the linking phase to the linker using -T option.

• Linker script command

- ENTRY
- MEMORY
- SECTIONS
- KEEP
- ALIGN
- AT >

• ENTRY command

- This command is used to set the "Entry point address" information in the header of final elf file generated.
- In our case "Reset_Handler" is the entry point into the application. The first piece of the code that execute right after the processor reset.
- The debugger uses this information to locate the first function to execute.
- Not a mandatory command to use but required when you use debug the elf file using debugger (GDB)
- Syntax
`ENTRY(symbol-name)`
 ex.
`ENTRY(Reset_Handler)`

Use of Entry command

- open command prompt & make .obj file
~ touch sim32-obj.o
- open file and write your linker script

ENTRY(Reset_Handler)

* Memory command

- This command allows you to describe the different memories present in the target and their start address and size information.
- The linker uses information mentioned in this command to assign addresses to merged sections.
- The information is given under this command also helps the linker to calculate total code and data memory consumed so far and throw an error message if data, code, heap or stack areas cannot fit into available size.

- By using memory command, you can fine-tune various memories available in your target and allow different sections to occupy different memory areas

- Typically one linker script has one memory command

- Memory command syntax

MEMORY

{

name(attr): ORIGIN = origin, LENGTH = len

}

define name of the memory region which will be later referenced by other parts of the linker script

define the length information

define origin address of memory region

Attribute letter	Meaning
R	Read only section
W	Read and write section
X	section contain executable code
A	Allocated section
I	Initialized section
L	Same as I
!	Invert the sense of any

MCU	Flash size kb	SRAM1 size	SRAM2 size
STM32F4VGT6	1024	112	16

• linker script code for memory

MEMORY

{

name (attr) : ORIGIN = origin, LENGTH = len

}

MEMORY

{

~~CODE~~ FLASH (rx)

* Section Command

- section command is used to create different output sections in the final elf executable generated.

- important command by which you can instruct the linker how to merge the input sections to yield an output section.

- This command also controls the order in which different output sections appear

in the elf file generated.

- By using this command, you also mention the placement of a section in a memory region.

ex. you instruct the linker to place the .text section in the FLASH memory region, which is described by the MEMORY command.

```
/* Section */
```

```
SECTIONS
```

```
{
```

```
/* This section should include .text */
```

```
.text:
```

```
{
```

```
    // merge all .isr_vector
```

```
    // merge all .text section
```

```
    // merge all .rodata
```

```
} >(vma) AT>(lma)
```

```
/* This section should include .data all input files
```

```
.data:
```

```
{
```

```
    // here merge all .data section all input files
```

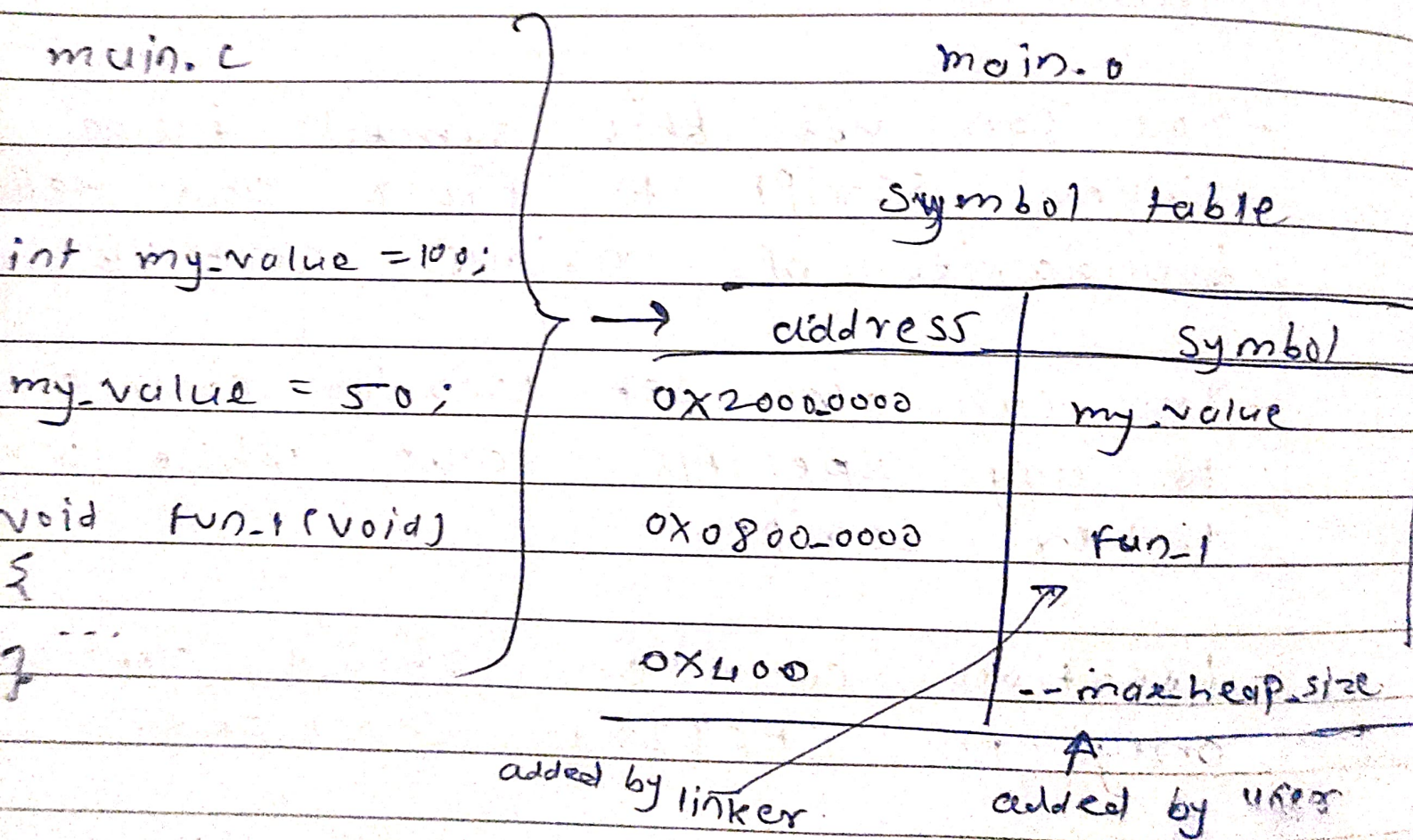
```
} >(vma) AT>(lma)
```

* Location counter

- This is a special linker symbol denoted by dot (.)
- This symbol is called 'location counter' since linker automatically update this symbol with location (address) information
- You can use this symbol inside the linker script to track and define boundaries of various sections.
- You can also set location counter to any specific value while writing linker script.
- Location counter should appear only in SECTION command.
- The location counter is incremented by the size of the output section,

Linker Script Symbol

- A Symbol is a name of an address.
- A Symbol declaration is not equivalent to a variable declaration what you do in your c application.



`--max-heap-size = 0x400;` // Symbol declaration in linker script